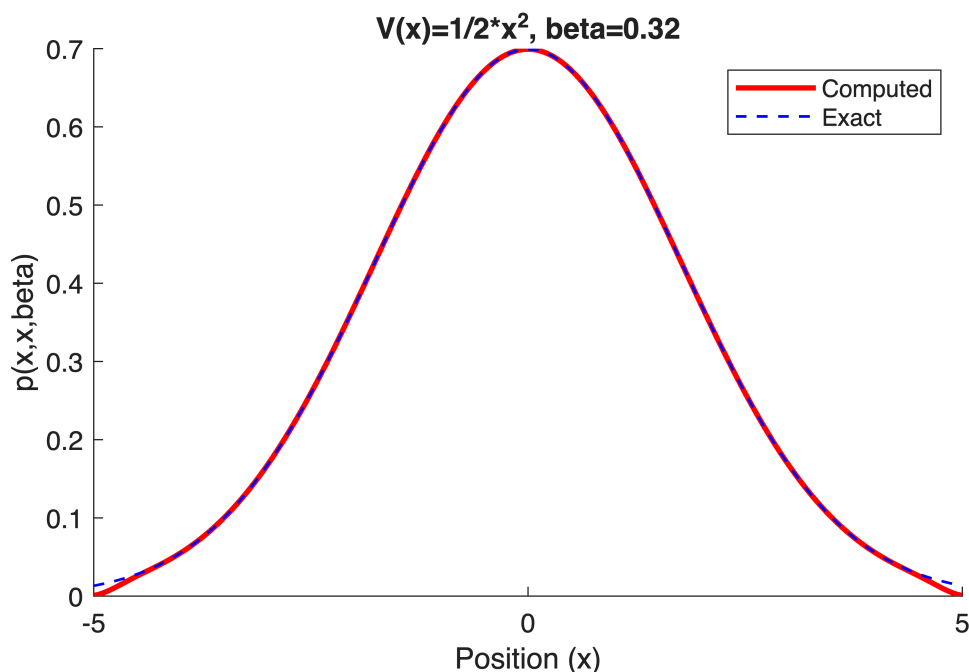


```

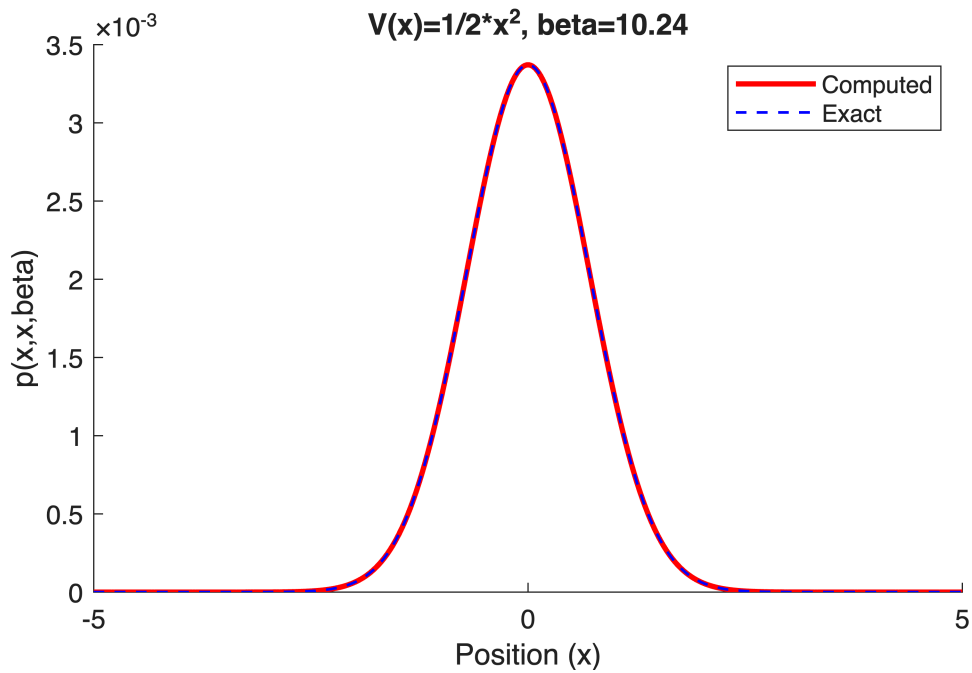
%Joseph Dearing
function matrixsquare(L,N,beta,n) %Length,#postions,beta,#repititions
%for simplicity m=h=1
xvec=linspace(-L,L,N)';
dx=2*L/(N-1);
[x,xdash]=meshgrid(xvec,xvec);
%trotter approximate of p with V(x)=1/2 * x^2
pfree=sqrt(1/(2*pi*beta))*exp(-(x-xdash).^2/(2*beta));
e1=exp(-0.5*beta*(0.5*x.^2));
e2=exp(-0.5*beta*(0.5*xdash.^2));
p1=e1.*pfree.*e2;
%incorporates both for loops of algorithm, repeats n times
for i = 1:n
    p1=dx*(p1*p1);
end
betaf=beta*2^n;
p1diag=diag(p1);
pdiagexact=sqrt(1/(2*pi*sinh(betaf)))*exp(-xvec.^2*tanh(betaf/2));
%diagonal density matrix of eq 6.24
figure;
hold on;
plot(xvec,p1diag,'r-','LineWidth',2)
plot(xvec,pdiagexact,'b--','LineWidth',1)
legend('Computed', 'Exact');
xlabel('Position (x)');
ylabel('p(x,x,beta)');
title(['V(x)=1/2*x^2, beta=',num2str(betaf)]);
end

```

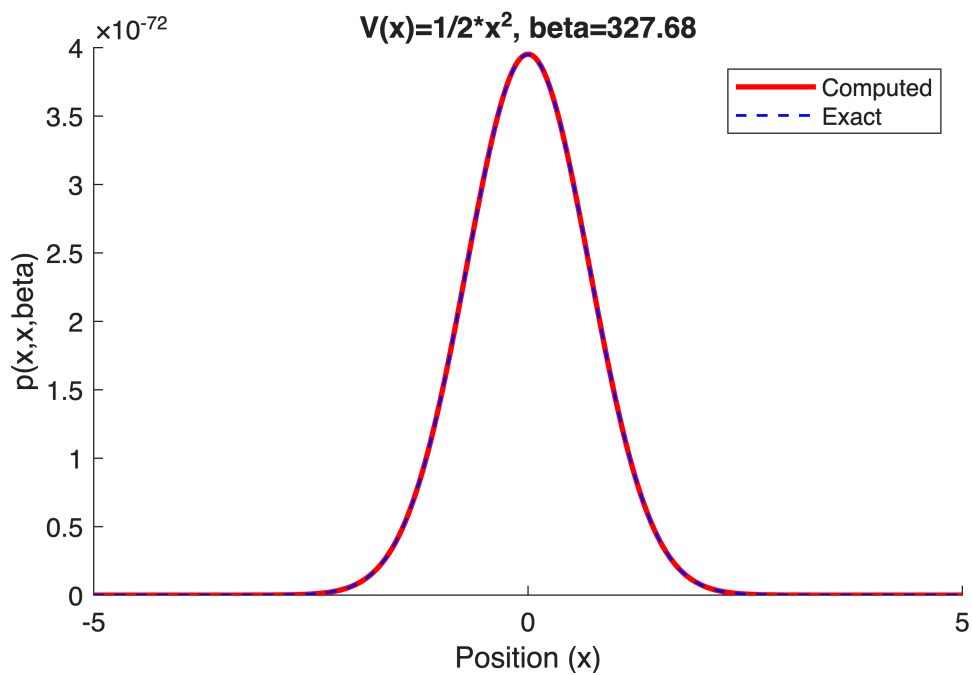
```
matrixsquare(5,300,0.01,5)
```



```
matrixsquare(5,300,0.01,10)
```



```
matrixsquare(5,300,0.01,15)
```



We can see that for various temperatures, we recover the diagonal density matrix of eq 6.24 ($\beta = 1/k_B T$)

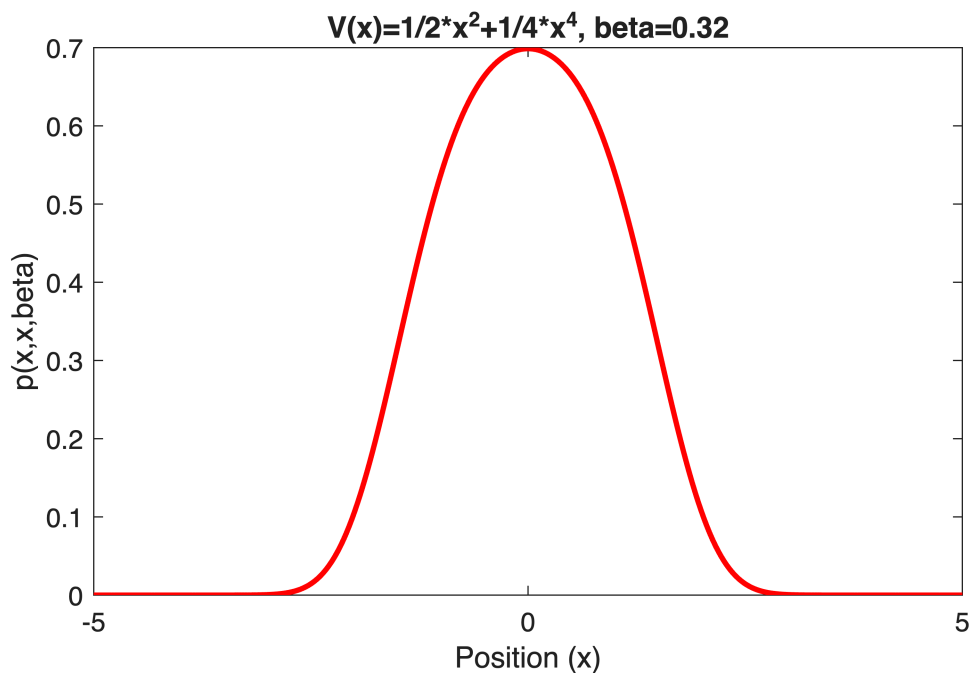
```
function matrixsquare2(L,N,beta,n) %Length,#postions,beta,#replititions
%for simplicity m=h=1
xvec=linspace(-L,L,N)';
dx=2*L/(N-1);
[x,xdash]=meshgrid(xvec,xvec);
%trotter approximate of p with V(x)=1/2 * x^2 + 1/4 * x^4
pfree=sqrt(1/(2*pi*beta))*exp(-(x-xdash).^2/(2*beta));
e1=exp(-0.5*beta*(0.5*x.^2+0.25*x.^4));
e2=exp(-0.5*beta*(0.5*xdash.^2+0.25*xdash.^4));
p1=e1.*pfree.*e2;
```

```

%incorporates both for loops of algorithm, repeats n times
for i = 1:n
    p1=dx*(p1*p1);
end
betaf=beta*2^n;
p1diag=diag(p1);
figure;
plot(xvec,p1diag,'r-','LineWidth',2)
xlabel('Position (x)');
ylabel('p(x,x,beta)');
title(['V(x)=1/2*x^2+1/4*x^4, beta=',num2str(betaf)]);
end

```

```
matrixsquare2(5,300,0.01,5)
```



```

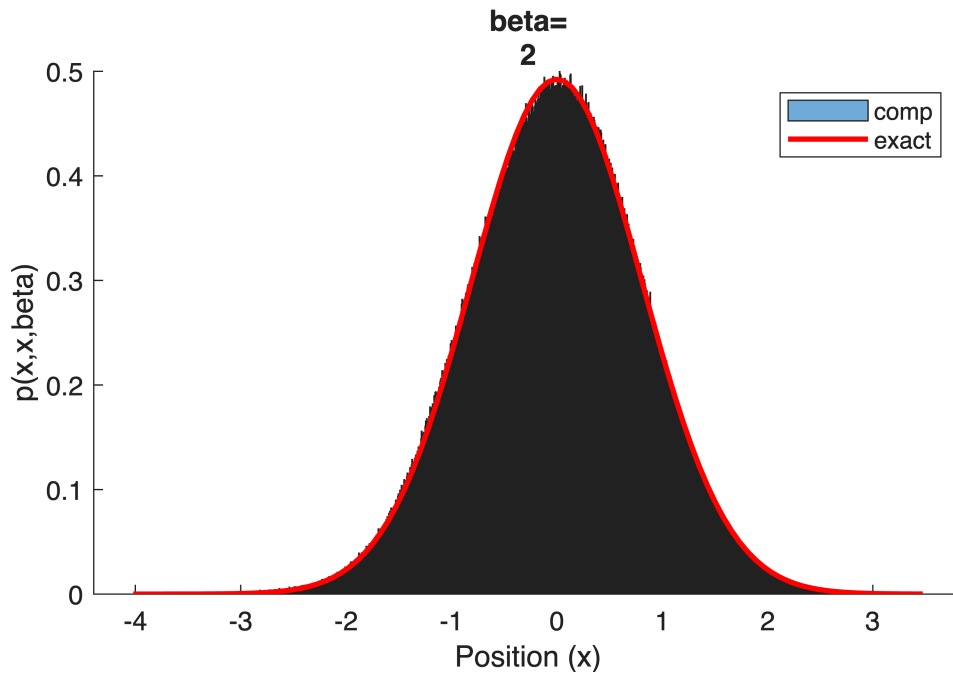
function xout = nhp(x,beta,delta,n) %Positions,beta,delta,#iterations
N=length(x);
xout=zeros(n,N);
dt=beta/N;
for i = 1:n
    k=randi(N);
    kplus=mod(k,N)+1; %adjusted for matlab indexing, ensuring if K = "N"
    K+ <- 1
    kminus=mod(k-2,N)+1; %adjusted for matlab indexing, ensuring if K =
    "0" K- <- "N-1"
    xdash=x(k)+(2*rand()-1)*delta;
    %prefactor of pfree cancels out when setting upsiion = pib/pia so have
    %removed from calculation, again setting m=h=1 for simplicity
    expa=(-(x(kminus)-x(k))^2 / (2*dt)) + (-(x(k)-x(kplus))^2 / (2*dt)) +
    (-0.5*dt*x(k)^2);
    expb=(-(x(kminus)-xdash)^2 / (2*dt)) + (-(xdash-x(kplus))^2 / (2*dt))
    + (-0.5*dt*xdash^2);
    upsiion=exp(expb-expa);
    if rand() < upsiion
        x(k) = xdash;
    end
    xout(i,:)=x;
end
end

```

```

N=20;
xin=linspace(-1,1,N);
nsteps=10000000;
beta=2;
samples=nhp(xin,beta,0.5,nsteps);
burn=round(0.2*nsteps);
samples=samples(burn:nsteps,:);
samples=samples(:)';
xplot=linspace(min(samples),max(samples),100);
pexact=sqrt(tanh(beta/2)/pi)*exp(-xplot.^2*tanh(beta/2)); %diagonal
density matrix of eq 6.24, normalized by dividing by Z(beta)
figure;
hold on;
histogram(samples,'Normalization','pdf');
plot(xplot,pexact,'r-','LineWidth',2);
xlabel('Position (x)');
ylabel('p(x,x,beta)');
legend('comp','exact');
title(["beta=",num2str(beta)]);

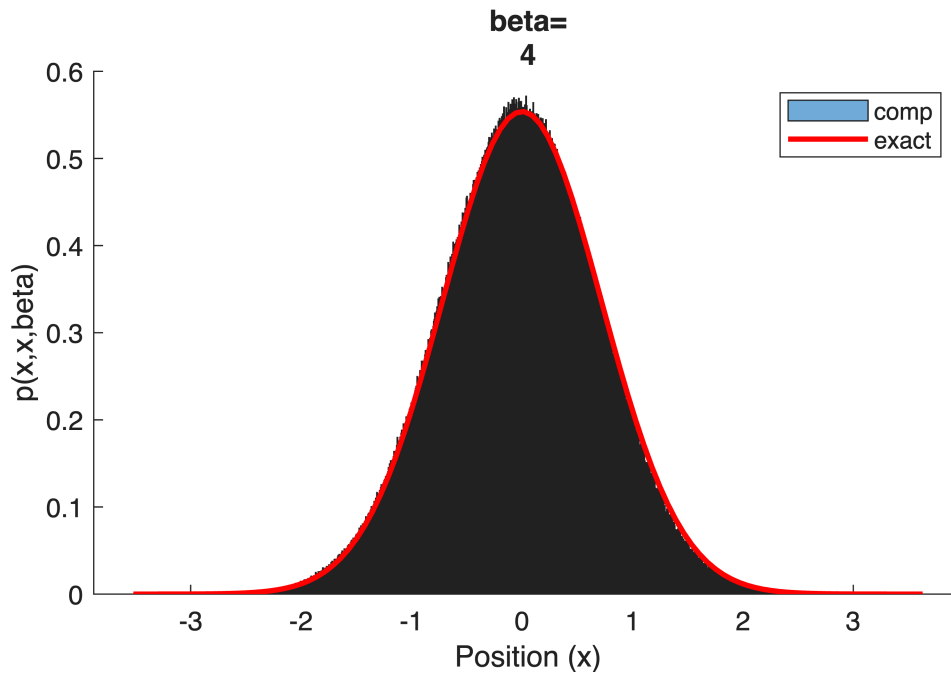
```



```

N=20;
xin=linspace(-1,1,N);
nsteps=10000000;
beta=4;
samples=nhp(xin,beta,0.5,nsteps);
burn=round(0.2*nsteps);
samples=samples(burn:nsteps,:);
samples=samples(:)';
xplot=linspace(min(samples),max(samples),100);
pexact=sqrt(tanh(beta/2)/pi)*exp(-xplot.^2*tanh(beta/2)); %diagonal
density matrix of eq 6.24, normalized by dividing by Z(beta)
figure;
hold on;
histogram(samples,'Normalization','pdf');
plot(xplot,pexact,'r-','LineWidth',2);
xlabel('Position (x)');
ylabel('p(x,x,beta)');
legend('comp','exact');
title(["beta=",num2str(beta)]);

```



We can see that the naive harmonic path algorithm recovers the analytical results of 6.24 at different temperatures

```
function xout = nhp2(x,beta,delta,n) %Positions,beta,delta,#iterations
N=length(x);
xout=zeros(n,N);
dt=beta/N;
for i = 1:n
    k=randi(N);
    kplus=mod(k,N)+1; %adjusted for matlab indexing, ensuring if K = "N"
    K+ <- 1
    kminus=mod(k-2,N)+1; %adjusted for matlab indexing, ensuring if K =
    "0" K- <- "N-1"
    xdash=x(k)+(2*rand()-1)*delta;
    %prefactor of pfree cancels out when setting upilon = pib/pia so have
    %removed from calculation, again setting m=h=1 for simplicity
    expa=(-(x(kminus)-x(k))^2 / (2*dt)) + (-(x(k)-x(kplus))^2 / (2*dt)) -
    dt*(0.5*x(k)^2+0.25*x(k)^4);
    expb=(-(x(kminus)-xdash)^2 / (2*dt)) + (-(xdash-x(kplus))^2 / (2*dt))
    - dt*(0.5*xdash^2+0.25*xdash^4);
    upilon=exp(expb-expa);
    if rand() < upilon
        x(k) = xdash;
    end
    xout(i,:)=x;
end
end
```

```
function [xvec,p1diag]=matrixsquare2(L,N,beta,n)
%Length,#postions,beta,#repititions
%for simplicity m=h=1
```

```

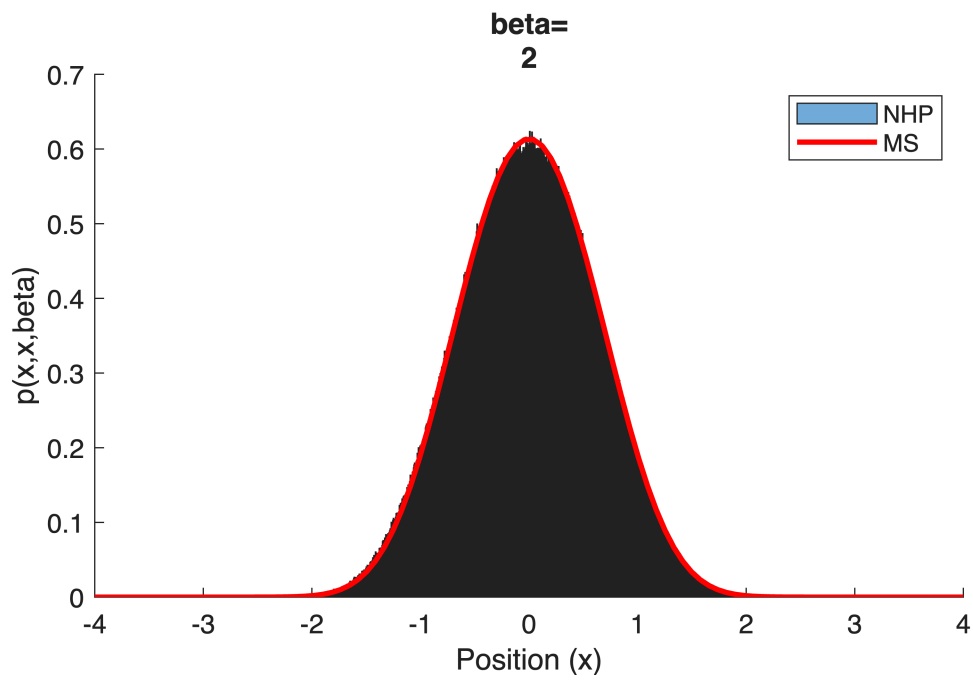
xvec=linspace(-L,L,N)';
dx=2*L/(N-1);
[x,xdash]=meshgrid(xvec,xvec);
%trotter approximate of p with  $V(x)=1/2 * x^2 + 1/4 * x^4$ 
pfree=sqrt(1/(2*pi*beta))*exp(-(x-xdash).^2/(2*beta));
e1=exp(-0.5*beta*(0.5*x.^2+0.25*x.^4));
e2=exp(-0.5*beta*(0.5*xdash.^2+0.25*xdash.^4));
p1=e1.*pfree.*e2;
%incorporates both for loops of algorithm, repeats n times
for i = 1:n
    p1=dx*(p1*p1);
end
p1diag=diag(p1);
figure;
end

```

```

N=20;
xin=linspace(-1,1,N);
nsteps=10000000;
beta=2;
samples=nhp2(xin,beta,0.5,nsteps);
burn=round(0.2*nsteps);
samples=samples(burn:nsteps,:);
samples=samples(:)';
nreps=4;
%L=4 chosen to be approx 5 times sd of p so it encompasses 99% of gaussian
%like distribution
[xvec,pdiag]=matrixsquare2(4,100,beta/(2^nreps),nreps);
Z=trapz(xvec,pdiag);
pnorm=pdiag/Z;
figure;
hold on;
histogram(samples,'Normalization','pdf');
plot(xvec,pnorm,"r-","LineWidth",2)
xlabel('Position (x)');
ylabel('p(x,x,beta)');
legend('NHP','MS');
title(["beta=",num2str(beta)]);

```



We can see that the naive harmonic path algorithm recovers the same diagonal density matrix as the matrix squaring algorithm for an anharmonic oscillator

```

function P = twocycles(N)
%Creating dictionary
Y=zeros(1,N+2);
Y(1)=0; %Y(-1)=0
Y(2)=1; %Y(0)=1
Y(3)=1; %Y(1)=1
for n = 2:N
    Y(n+2)=Y(n+1)+(n-1)*Y(n); %recurrence relation 7.6 with adjusted
    indexing
end

Q=randperm(N);
M=N;
P={};
while M>0
    epsilon=rand()*Y(M+2); %adjusted for matlab indexing
    if epsilon<Y(M+1)
        P{end+1}=Q(M);
        M=M-1;
    else
        P{end+1}=[Q(M),Q(M-1)];
        M=M-2;
    end
end
end

```

twocycles(8)

ans = 1x7 cell

	1	2	3	4	5	6	7
1	1	3	6	4	[8,5]	7	2

twocycles(10)

ans = 1x6 cell

	1	2	3	4	5	6
1	[1,2]	[7,9]	[5,10]	4	[6,3]	8

The two-cycles algorithm is a sampling method for generating a random permutation with N elements consisting of cycles either length 1 or length 2. It works by shuffling the N element into order Q . Then it takes the M th element of Q decides whether that element stays a one cycle, or if its paired up with the $(M-1)$ th element of Q to form a two cycle. Upon which decision is made, M is updated accordingly. The testing stage determines such decision through the use of ϵ . Upon entering the testing stage ($M=N$), we have $Y(M)$ total outcomes for the rest of the sampling. If the element $Q(M)$ forms a 1-cycle we then we now have $Y(M-1)$ possible outcomes. This implies that the probability that $Q(M)$ forms a 1-cycle is $Y(M-1)/Y(M)$. Therefore, the algorithm tests against such threshold by generating the random number ϵ and then decreases M accordingly. The dictionary is good as it allows for direct sampling, meaning we never have to reject a choice, optimizing efficiency.

```

function P = threecycles(N)
%creating dictionary
%recurrence relation becomes  $Y(N)=Y(N-1)+(N-1)Y(N-2)+(N-1)(N-2)Y(N-3)$ 
%Hence, we need also to define  $Y(-2)=0$ 
Y=zeros(1,N+3);
Y(1)=0; %Y(-2)=0
Y(2)=0; %Y(-1)=0
Y(3)=1; %Y(0)=1
Y(4)=1; %Y(1)=1
for n = 2:N
    Y(n+3)=Y(n+2)+(n-1)*Y(n+1)+(n-1)*(n-2)*Y(n); %adjusted indexing for
matlab
end

Q=randperm(N);
M=N;
P={};
while M>0
    %testing phase updated to compare epsilon against the same threshold
    %for 1-cycles and a new threshold determining whether or not to
    categorize as two cycles
    epsilon=rand()*Y(M+3);
    if epsilon<Y(M+2)
        P{end+1}=Q(M);
        M=M-1;
    elseif epsilon<Y(M+2)+(M-1)*Y(M+1)
        P{end+1}=[Q(M),Q(M-1)];
        M=M-2;
    else
        P{end+1}=[Q(M),Q(M-1),Q(M-2)];
        M=M-3;
    end
end
end
end

```

threecycles(8)

ans = 1×3 cell

	1	2	3
1	[1,4,5]	[8,2]	[3,6,7]

threecycles(10)

ans = 1×5 cell

	1	2	3	4	5
1	10	[5,8,2]	6	[1,9]	[4,7,3]

```
function z=calcz(N,beta)
k=1:N;
z=(1./(1-exp(-k.*beta))).^3;%equation 7.18
end
```

```
function Z=canonicrecursion(z)
N=length(z);
Z=zeros(1,N+1);
Z(1)=1;
for M = 1:N
    Z(M+1)=sum(Z(1:M).*z(M:-1:1))/M;
end
end
```

```
function m=directcycles(Z,z)
N=length(z);
m=zeros(1,N);
M=N;
while M>0
    kset=z(1:M).*Z(M:-1:1);%set from which to sample k
    kset=kset/sum(kset);%normalizing for sampling
    k=find(cumsum(kset)>=rand(),1,'first');
    M=M-k;
    m(k)=m(k)+1;
end
end
```

```
%as given on webiste, edited to work in matlab
function x=lharmonic(dtau,N)
beta=N*dtau;
x=zeros(1,N);
x(1)=randn()/sqrt(2*tanh(beta/2));
for k = 1:N-1
    U1=1/tanh(dtau)+1/tanh((N-k)*dtau);
    U2=x(k)/sinh(dtau)+x(1)/sinh((N-k)*dtau);
    xmean=U2/U1;
    sigma=1/sqrt(U1);
    x(k+1)=xmean+sigma*randn();
end
end
```

```
function x=dhb(z,Z,dtau)
N=length(z);
x=zeros(1,N);
m=directcycles(Z,z);
l=0;
for k=1:N
    if m(k)>0
        for i=1:m(k)
            x(l+1:l+k)=lharmonic(dtau,k);
```

```

        l=l+k;
    end
end
end
end

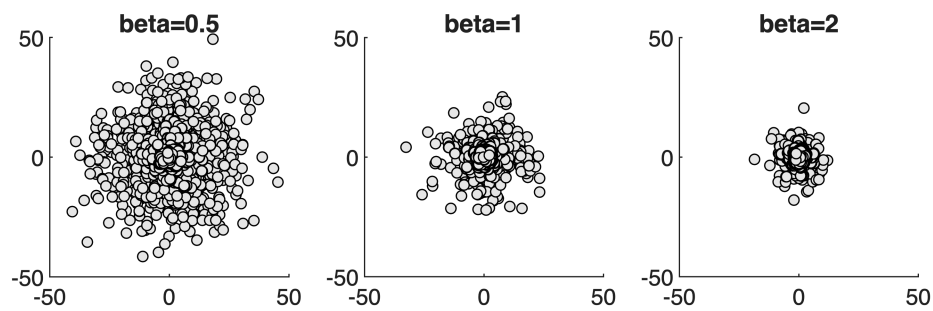
```

Bosons are indistinguishable, so we must account for particles exchanging with one another. Algorithm dhb takes as input z , Z and $d\tau$. z represents the statistical weight of a particle forming a loop of length k . We then use a recurrence relation to calculate the partition function Z . With these values we use direct cycles to determine what loop size each particle belongs to in a probabilistic manner. Once we have determined the cycle structure we use the levy harmonic algorithm to determine their positions

```

N=100;
nevents=50;
betas=[0.5,1,2];
figure;
for j =1:3
    beta=betas(j);
    dtau=beta/N;
    z=calcz(N,beta);
    Z=canonicrecursion(z);
    x=zeros(nevents,N);
    y=zeros(nevents,N);
    for i=1:nevents
        x(i,:)=dhb(z,Z,dtau);
        y(i,:)=dhb(z,Z,dtau);
    end
    subplot(1,3,j)
    scatter(x(:),y(:),15,'MarkerEdgeColor','k','MarkerFaceColor',[0.9 0.9
0.9]);
    title(['beta=',num2str(beta)]);
    axis square;
    xlim([-50 50]);
    ylim([-50 50]);
end

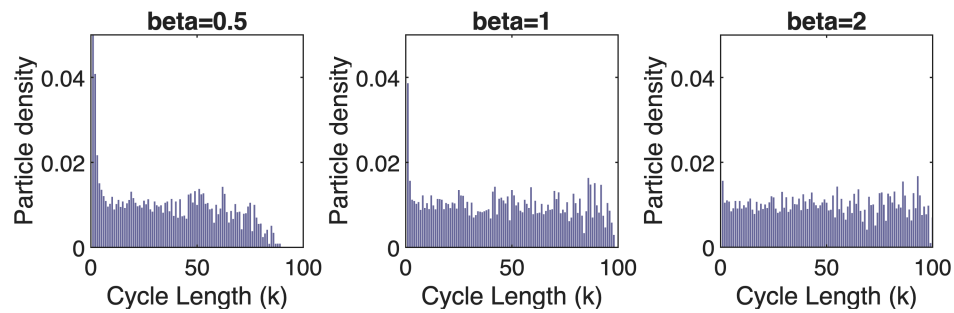
```



```

N=100;
nevents=1000;
betas=[0.5,1,2];
figure;
for j=1:3
    beta=betas(j);
    z=calcz(N,beta);
    Z=canonicrecursion(z);
    mout=zeros(1,N);
    for i=1:nevents
        m=directcycles(Z,z);
        mout=mout+m;
    end
    mavg=mout/nevents;
    k=1:N;
    cycleweight=(k.*mavg)/N;
    subplot(1, 3, j)
    bar(k, cycleweight, 'FaceColor', [0.4 0.4 0.6], 'EdgeColor', 'none');
    title(['beta=', num2str(beta)]);
    xlabel('Cycle Length (k)');
    ylabel('Particle density');
    axis square;
    xlim([0 N]);
    ylim([0 0.05])
end

```

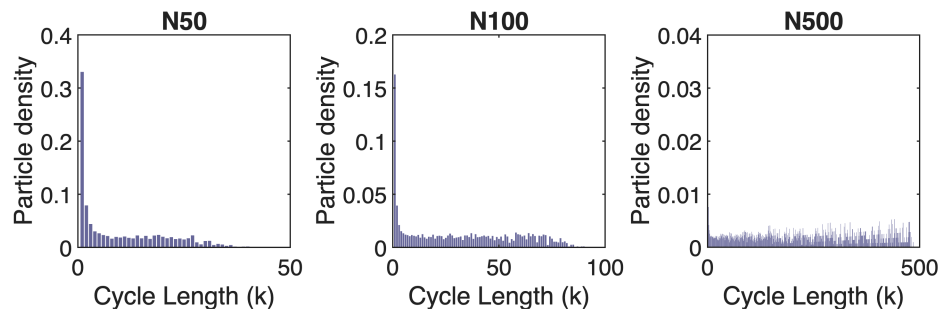


For high temp (small beta) the distribution is dominated by very short cycles. As we increase beta this spike eventually 'flattens' out.

```

Ns=[50,100,500];
nevents=1000;
beta=0.5;
figure;
for j=1:3
    N=Ns(j);
    z=calcz(N,beta);
    Z=canonicrecursion(z);
    mout=zeros(1,N);
    for i=1:nevents
        m=directcycles(Z,z);
        mout=mout+m;
    end
    mavg=mout/nevents;
    k=1:N;
    cycleweight=(k.*mavg)/N;
    subplot(1, 3, j)
    bar(k, cycleweight, 'FaceColor', [0.4 0.4 0.6], 'EdgeColor', 'none');
    title(['N', num2str(N)]);
    xlabel('Cycle Length (k)');
    ylabel('Particle density');
    axis square;
    xlim([0 N]);
end

```



We see the same trend as before, where the distribution of the cycle lengths is dominated by small loops for small N , and as N increases the distribution flattens out. These graphs show that for a temperature that is too hot for condensation to occur for a set amount of particles is actually sufficient enough if we increase N accordingly. So as N tends to infinity the critical temperature also approaches infinity.